



Estrela Gestão

Guia de instalação local em servidor (Docker) —
Ubuntu/Debian

PostgreSQL 16 + FastAPI/Gunicorn + Caddy · 100% local e offline
Gerado em 13/07/2026

Conteúdo

- 02** Visão geral e requisitos
- 03** Preparar o servidor (Ubuntu/Debian)
- 04** Baixar o sistema
- 05** Configurar os segredos (.env.prod)
- 06** Subir a stack
- 07** Configurar o acesso na rede (DNS + HTTPS)
- 08** Criar o admin e trocar senhas
- 09** Importar os dados reais (ETL)
- 10** Agendar backup diário
- 11** Manutenção remota (Tailscale)
- 12** Configurar os 10 terminais (PWA)
- 13** Verificação final (go-live)
- 14** Operação do dia a dia (resumo)
- 15** Troubleshooting

Guia de Instalação Local em Servidor (Docker)

Passo a passo para deixar o **Estrela Gestão** rodando em uma máquina servidor (mini PC), em containers Docker, acessado pelos terminais em modo aplicativo (PWA). Alvo: **Ubuntu Server / Debian**. Tudo 100% local — não depende de internet para operar.

1. Visão geral e requisitos

O Estrela Gestão roda como **três containers** na máquina servidor:

Container	Imagem	Função
db	postgres:16.4	Banco de dados (PostgreSQL 16). Guarda tudo, inclusive as fotos dos produtos.
app	construída do Dockerfile	A aplicação (FastAPI + Gunicorn). Aplica as migrations e sobe em 0.0.0.0:8000.
caddy	caddy:2	Proxy reverso com HTTPS interno automático para https://sistema.local.

Acesso dos terminais: https://sistema.local no navegador, instalável como aplicativo. Manutenção remota só por Tailscale (fora dos containers).

Requisitos do servidor:

- **Sistema:** Ubuntu Server 22.04 ou 24.04 LTS (ou Debian 12). Atualizado.
- **Hardware (mínimo recomendado):** 4 vCPU, 8 GB RAM, 120 GB SSD. O banco cresce com fotos (bytea) e histórico de movimentações — reserve espaço.
- **Rede:** IP **fixo** na LAN (não DHCP). Os terminais acessam por https://sistema.local, que precisa apontar para esse IP (DNS ou hosts).
- **Internet:** só para instalar Docker, baixar o sistema e (opcional) backup offsite. Em operação normal, nada depende de internet.
- **No-break:** recomendado, para a stack reiniciar sozinha após falta de luz (restart: always).

Recomendação de simplicidade: use uma máquina dedicada só para o sistema. Não instale outros serviços que possam ocupar as portas 80/443.

2. Preparar o servidor (Ubuntu/Debian)

2.1 Atualizar e ajustar timezone

```
sudo apt update && sudo apt upgrade -y
sudo timedatectl set-timezone America/Sao_Paulo
```

2.2 Definir IP fixo

Configure a interface de rede com IP fixo (ex.: 192.168.1.50) no roteador (reserva de DHCP) ou no Netplan do servidor. Anote o IP — ele vira `sistema.local` no passo 6.

2.3 Criar um usuário para o sistema (não rodar como root)

```
sudo adduser estrela
sudo usermod -aG sudo estrela      # permite administrar o servidor
su - estrela
```

2.4 Instalar Docker Engine + plugin Compose

Use o instalador oficial (baixa e configura o repositório da Docker):

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker estrela    # permite rodar docker sem sudo
```

Encerre a sessão e entre de novo (para o grupo `docker` valer) e valide:

```
docker run --rm hello-world
docker compose version
```

Recomendação de simplicidade: o instalador `get.docker.com` cuida do repositório, do daemon e do plugin `compose` num único comando. Evita instalar o antigo `docker-compose` (Python).

2.5 Garantir que o Docker sobe no boot

```
sudo systemctl enable --now docker
```

3. Baixar o sistema

Coloque o projeto em `/opt/estrela` (o runbook e os scripts de backup já assumem esse caminho).

Se o servidor tem internet:

```
sudo mkdir -p /opt/estrela
sudo chown estrela:estrela /opt/estrela
cd /opt/estrela
git clone <url-do-repositorio> .    # ou: git clone ... /opt/estrela
```

Se o servidor está offline (mais comum no cliente): gere um tarball na máquina de desenvolvimento e copie por pen drive/SCP:

```
# na máquina de desenvolvimento:
tar --exclude='.venv' --exclude='__pycache__' --exclude='tailwindcss' \
  -czf estrela.tar.gz -C /caminho/estrela_softwarelocal .

# no servidor:
```

```
sudo mkdir -p /opt/estrela && sudo chown estrela:estrela /opt/estrela
cd /opt/estrela
tar -xzf /mnt/pen/estrela.tar.gz
```

Confirme que estão presentes:

```
ls -l docker-compose.prod.yml Dockerfile Caddyfile scripts/entrypoint.sh .env.prod.example
```

4. Configurar os segredos (`.env.prod`)

```
cd /opt/estrela
cp .env.prod.example .env.prod
```

Edite `.env.prod` e preencha **somente** estas variáveis:

Variável	Valor	Como gerar
<code>DB_PASSWORD</code>	senha forte do PostgreSQL	<code>openssl rand -base64 24</code>
<code>JWT_SECRET</code>	segredo do token (mín. 32 caracteres)	<code>openssl rand -hex 32</code>
<code>JWT_EXPIRES_MIN</code>	expiração do login	<code>480</code> (8 h)
<code>ENV</code>	ambiente	<code>prod</code>

Gere os dois segredos no terminal e cole no arquivo:

```
echo "DB_PASSWORD=$(openssl rand -base64 24)"
echo "JWT_SECRET=$(openssl rand -hex 32)"
```

Atenção:

- **Não** coloque `DATABASE_URL` no `.env.prod`. O `docker-compose.prod.yml` já monta a URL a partir do `DB_PASSWORD` (`...@db:5432/estrela_gestao`). Definir `DATABASE_URL` manualmente pode apontar para o host errado.
- As variáveis `S3_*` (MinIO) podem **ficar vazias**. As fotos dos produtos hoje moram no Postgres (coluna `bytea`), não no MinIO. O MinIO é legado e só aparece em migrations antigas.
- Em `ENV=prod`, o **startup recusa** `JWT_SECRET` **fraco** (curto demais ou igual a um valor de exemplo). Se a app reiniciar em loop, é quase sempre isso — veja `docker compose logs app`.
- `.env.prod` **não é versionado** (está no `.gitignore`). Nunca o envie para o repositório.

Recomendação de simplicidade: tudo que muda entre clientes/ambientes vive em `.env.prod`. Não edite o `docker-compose.prod.yml` para mudar senhas — troque só o `.env.prod`.

5. Subir a stack

Para não digitar o comando longo toda vez, crie um **alias** no `~/.bashrc`:

```
echo 'export COMPOSE="docker compose -f docker-compose.prod.yml --env-file .env.prod"' >>
~/.bashrc
source ~/.bashrc
cd /opt/estrela
```

Suba tudo (db, app, caddy):

```
$COMPOSE up -d --build
```

O `--build` monta a imagem do `app` na primeira vez (demora alguns minutos). Depois, para subir normal, basta `$COMPOSE up -d`.

Acompanhe o estado:

```
$COMPOSE ps          # ver db/app/caddy e seus healthchecks
$COMPOSE logs -f app  # acompanhar o startup do app
```

O que esperar no `entrypoint` do `app`:

```
[entrypoint] Aplicando migrations (alembic upgrade head)...
[entrypoint] Iniciando Unicorn (3 workers Unicorn) em 0.0.0.0:8000...
```

As migrations (criação das tabelas) rodam **automaticamente** no boot do container — não é preciso rodar `alembic` à mão no fluxo normal. O `app` só sobe depois do `db` ficar saudável (`depends_on: service_healthy`).

Verifique o endpoint de saúde (de dentro da rede do servidor):

```
docker compose -f docker-compose.prod.yml exec app curl -s http://localhost:8000/health
# {"status":"ok"}
```

Recomendações de simplicidade / robustez: - `restart: always` em todos os serviços: a stack volta sozinha após reinício do mini PC. - O healthcheck do `db` + `depends_on` garante a ordem de boot (db antes do app). - (Opcional) adicione rotação de logs no `docker-compose.prod.yml` para não encher o disco: `logging: { driver: json-file, options: { max-size: "10m", max-file: "3" } }` em cada serviço.

6. Configurar o acesso na rede (DNS + HTTPS interno)

6.1 Fazer `sistema.local` apontar para o servidor

Escolha **uma** das opções:

Opção A — `hosts` em cada terminal (simples, sem infra): edite o arquivo de hosts em cada terminal (Windows: `C:\Windows\System32\drivers\etc\hosts` ; Linux: `/etc/hosts`) e adicione:

```
192.168.1.50  sistema.local
```

(substitua pelo IP fixo do servidor)

Opção B — mDNS/Avahi no servidor (automático na rede): instale e publique o nome:

```
sudo apt install -y avahi-daemon
sudo sed -i 's/#domain-name=.*domain-name=local/' /etc/avahi/avahi-daemon.conf
sudo systemctl restart avahi-daemon
```

Assim `sistema.local` resolve em qualquer terminal sem editar `hosts`.

6.2 Confiar no certificado interno do Caddy

O Caddy gera um certificado com uma **CA local** (`tls internal`). Por padrão os navegadores mostram aviso. Para resolver de vez, instale a CA do Caddy nos terminais:

1. No servidor, copie a CA para fora do container (já gerada após o primeiro boot do Caddy): `bash docker cp $(docker compose -f /opt/estrela/docker-compose.prod.yml ps -q caddy):/data/caddy/pki/authorities/local/root.crt /tmp/estrela-root-ca.crt`
2. Distribua `estrela-root-ca.crt` para os terminais (pen drive, rede) e importe como **Autoridade de Certificação raiz confiável** no sistema/navegador de cada terminal.
3. Recarregue o navegador. O `https://sistema.local` passa a abrir sem aviso.

Recomendação de simplicidade: a opção A (`hosts`) é a mais rápida para 10 terminais; instale a CA do Caddy uma única vez por terminal e os 10 ficam OK por anos.

O `Caddyfile` atual (já no projeto):

```
sistema.local {
    reverse_proxy app:8000
    tls internal
}
```

7. Criar o admin e trocar senhas

Abra `https://sistema.local` no navegador. Para o **primeiro acesso**, crie os usuários iniciais:

Opção A — rodar o seed (cria 1 usuário por perfil, dados de exemplo):

```
$COMPOSE exec app python scripts/seed.py
```

Isso cria `admin@estrela.local`, `vendedor@estrela.local`, `financeiro@estrela.local`, `funcionario@estrela.local` — todos com a senha `estrela123` (senha de dev, fraca).

Troque a senha do admin imediatamente após o primeiro login (a política de senha forte do sistema, em `security.py`, exige mínimo de 10 caracteres e classes variadas). O seed é só para bootstrap; em produção, cadastre os usuários reais pela tela `/usuarios`.

Opção B — cadastrar pela tela: logue como admin (após o seed), vá em **Usuários** e crie os usuários reais por perfil. Prefira esta opção para produção.

Login de dev/admin (após o seed):

```
E-mail: admin@estrela.local
Senha: estrela123
```

8. Importar os dados reais (ETL da planilha)

Suba a planilha real (`CONTROLE.xlsx`) para o servidor (ex.: `/opt/estrela/data/`):

```
# da máquina de desenvolvimento para o servidor:
scp data/CONTROLE.xlsx estrela@192.168.1.50:/opt/estrela/data/
```

Sempre rode primeiro em modo de validação (`--dry-run`): lê a planilha, valida, e gera o `relatorio_inconsistencias.xlsx` **sem gravar** no banco:

```
$COMPOSE exec app uv run python scripts/import_planilhas.py \
--file data/CONTROLE.xlsx --dry-run
```

Abra o `relatorio_inconsistencias.xlsx` (copie para fora do container se necessário), revise com o cliente e corrija a planilha. Repita o `--dry-run` até não haver inconsistências críticas.

Para gravar de fato (o importador é **idempotente** — rodar 2x não duplica):

```
$COMPOSE exec app uv run python scripts/import_planilhas.py --file data/CONTROLE.xlsx
```

O relatório de inconsistências é salvo no diretório de trabalho do container (`/app`). Para copiá-lo para o host: `docker cp $(docker compose ps -q app):/app/relatorio_inconsistencias.xlsx .`

9. Agendar backup diário

O backup é **no host** (não no container) via `pg_dump`, com rotação de 14 dias.

9.1 Criar o diretório de backup e preparar os scripts

```
sudo mkdir -p /backup/estrela_gestao
sudo chown estrela:estrela /backup/estrela_gestao
chmod +x /opt/estrela/scripts/backup-estrela.sh /opt/estrela/scripts/restore-estrela.sh
```

O `backup-estrela.sh` usa `DB_CONTAINER=estrela_softwarelocal-db-1` por padrão. Confira o nome real com `docker ps --format '{{.Names}}'` e ajuste a variável se precisar: `bash export DB_CONTAINER=$(docker ps --format '{{.Names}}' | grep -i db)`

9.2 Agendar no cron

```
crontab -e
```

Adicione (backup todo dia às 02:00, com retenção de 14 dias):

```
0 2 * * * /opt/estrela/scripts/backup-estrela.sh >> /var/log/estrela-backup.log 2>&1
```

9.3 Testar um backup na hora

```
/opt/estrela/scripts/backup-estrela.sh  
ls -lh /backup/estrela_gestao/
```

9.4 (Opcional) Backup offsite

Se o local tem internet, use `scripts/backup-offsite.sh` (rclone, criptografado) no cron às 02:30:

```
30 2 * * * /opt/estrela/scripts/backup-offsite.sh >> /var/log/estrela-offsite.log 2>&1
```

Ele só roda se houver conexão (faz ping antes). Sem internet, ignore.

Recomendação de simplicidade: teste o **restore** (`restore-estrela.sh`) numa máquina isolada antes de confiar no backup. Backup não testado não é backup.

10. Manutenção remota (Tailscale)

Para acessar o servidor a distância (sem expor portas à internet), instale o **Tailscale no host** (nunca dentro do container):

```
curl -fsSL https://tailscale.com/install.sh | sh  
sudo tailscale up
```

Boas práticas:

- Mantenha a **ACL** do Tailnet restrita (só quem precisa acessa).
- **Nunca abra portas do roteador** para a internet — o sistema é local por projeto.
- Use o SSH do Tailscale para manutenção (logs, restart, atualizações).

11. Configurar os 10 terminais (PWA)

Em cada terminal, com o navegador Chrome ou Edge:

1. Abra `https://sistema.local` (a CA do Caddy já confiável — passo 6.2).
2. Faça login com o perfil do usuário daquele terminal.
3. Instale como aplicativo: - **Chrome:** menu `:` → **Instalar Estrela Gestão...** (ou "Criar atalho" → marcar "Abrir como janela"). - **Edge:** menu `...` → **Aplicativos** → **Instalar este site como um aplicativo.** - **Atalho de desktop (linha de comando):** `chrome --app=https://sistema.local`

4. Coloque o ícone na área de trabalho / início do terminal.

O sistema abre em **tela cheia, sem barra do navegador** ("modo programa"). A atualização é **centralizada no servidor** (service worker network-first) — os terminais nunca precisam reinstalar nada.

Recomendação de simplicidade: configure um terminal inteiro (CA + atalho + login) e use o perfil dele como modelo. Demais terminais: só importam a CA e copiam o atalho.

12. Verificação final (go-live)

Checklist antes de considerar pronto:

- [] `$COMPOSE ps` mostra `db`, `app` e `caddy` saudáveis (sem `restart` em loop).
- [] `https://sistema.local` abre **sem aviso** de certificado (CA instalada).
- [] Login funciona; senha do admin foi trocada.
- [] ETL rodou (`--dry-run` limpo + carga) e o estoque aparece na tela de Estoque.
- [] Backup rodou à mão e gerou arquivo não vazio em `/backup/estrela_gestao/`.
- [] Pelo menos um terminal abre o sistema como **aplicativo** (ícone, tela cheia).
- [] Tailscale conectado no mini PC; ACL restrita; roteador não expõe portas.
- [] No-break instalado; `restart: always` confirmado (testar desligar/ligar o mini PC).

13. Operação do dia a dia (resumo)

```
cd /opt/estrela
# Subir / derrubar
$COMPOSE up -d
$COMPOSE down
# Ver logs
$COMPOSE logs -f app
# Estado
$COMPOSE ps
```

Atualizar versão (nova entrega do sistema):

```
cd /opt/estrela
git pull                # ou: tar -xzf nova-versao.tar.gz
$COMPOSE up -d --build  # recompõe a imagem e roda migrations
```

Rollback (voltar à versão anterior): retorne o código ao commit/tag anterior e suba de novo. Se uma migration precisar ser desfeita, use `alembic downgrade -1` dentro do container:

```
$COMPOSE exec app alembic downgrade -1
```

Detalhes completos em `docs/runbook-servidor.md`.

14. Troubleshooting

Sintoma	Causa provável	Solução
<code>app</code> reinicia em loop	<code>JWT_SECRET</code> fraco/curto (startup recusa em <code>ENV=prod</code>)	Gere com <code>openssl rand -hex 32</code> , atualize <code>.env.prod</code> , <code>\$COMPOSE up -d</code>
<code>app</code> reinicia em loop (outra causa)	<code>DB_PASSWORD</code> divergente entre <code>db</code> e <code>app</code> , ou banco não subiu	Confira <code>.env.prod</code> , rode <code>\$COMPOSE logs db</code> e <code>\$COMPOSE logs app</code>
502 Bad Gateway no navegador	<code>app</code> ainda subindo (ou travado)	<code>\$COMPOSE ps</code> e <code>\$COMPOSE logs app</code> ; aguarde ou reinicie o <code>app</code>
"Sua conexão não é particular"	CA do Caddy não confiável no terminal	Instale <code>estrela-root-ca.crt</code> como raiz confiável (passo 6.2)
<code>sistema.local</code> não resolve	DNS/hosts não aponta para o IP do servidor	Adicione linha em <code>/etc/hosts</code> ou configure Avahi (passo 6.1)
Porta 80/443 já em uso	Outro serviço no host ocupando	Pare o serviço ou mude as portas do Caddy no <code>docker-compose.prod.yml</code>
Migrations falham no boot	Banco inconsistente / versão antiga	<code>\$COMPOSE logs app</code> ; corrija e, se preciso, restaure backup
Senha do admin perdida	—	Recrie via <code>seed.py</code> (cria <code>admin@estrela.local</code> / <code>estrela123</code> se não existir) ou por <code>psql</code> : <code>\$COMPOSE exec db psql -U estrela -d estrela_gestao</code>

Dica geral: o primeiro lugar a olhar é sempre `$COMPOSE logs app`. Quase todo problema de startup aparece ali em texto plano.